Joining tables + normal forms

Practical examples first. Join logic next. Assignment prompts at the end.

Session focus

- Use simple examples to see 1NF, 2NF, and 3NF clearly.
- Use one messy user-data table as the live normalization case.
- Use joins to answer questions from related tables.
- Transfer the same join logic into the assignment schema.

Key idea

Store facts in the right place. Combine them again with joins when a question needs more than one table.

messy_user_data

user_id	full_name	phone_numbers	emails
1	Anna Jensen	12 34 56 78, 87 65 43 21	anna@x.dk, aj@work.dk
2	Noah Berg	22 33 44 55	noah@x.dk

What to look for

Repeated values, mixed responsibilities, values packed into one field, and answers that need more than one table.

1NF — Atomic values

1. First Normal Form (1NF): Eliminating Duplicate Records
A table is in 1NF if it satisfies the following conditions:

- All columns contain atomic values (i.e., indivisible values).
- Each row is unique (i.e., no duplicate rows).
- Each column has a unique name.
- The order in which data is stored does not matter.

Messy table

Address data stored as one combined value

user_data

user_id	full_name	email	phone	home_address	work_address	school_name	school_address	department_name
1	Anna Jensen	anna@school.dk	12345678	Nørregade 12, 4200 Slagelse	Industrivej 8, 4100 Ringsted	Zealand Academy	Bredahlsgade 1, 4200 Slagelse	Software Development
2	Mads Larsen	mads@school.dk	23456789	Parkvej 4, 4000 Roskilde	Havnevej 19, 4300 Holbæk	Zealand Academy	Bredahlsgade 1, 4200 Slagelse	IT Operations

Problem: home_address, work_address, and school_address each contain several facts in one column.

1NF version

The same table, but address data is split into atomic columns

user_data_1nf

user_id	full_name	email	phone	home_street	home_nr	home_postnr	home_city	work_street	work_nr	work_postnr	work_city	school_name	school_street	school_nr	school_postnr	school_city	department_name
1	Anna Jensen	anna@school.dk	12345678	Nørregade	12	4200	Slagelse	Industrivej	8	4100	Ringsted	Zealand Academy	Bredahlsgade	1	4200	Slagelse	Software Development
2	Mads Larsen	mads@school.dk	23456789	Parkvej	4	4000	Roskilde	Havnevej	19	4300	Holbæk	Zealand Academy	Bredahlsgade	1	4200	Slagelse	IT Operations

Result: the table is still one table, but each column now holds one value only.

1NF = atomic values

2NF — Remove partial dependencies

A relation is in 2NF if it satisfies the conditions of 1NF and additionally. No partial dependency exists, meaning every non-prime attribute (non-key attribute) must depend on the entire primary key, not just a part of it.

Messy table

Primary key: (user_id, address_type)

user_addresses_1nf

user_id	address_type	full_name	email	phone	street_name	street_no	postal_code	city	school_name	school_street_name	school_street_no	school_postal_code	school_city	department_name
1	home	Anna Jensen	anna@school.dk	12345678	Nørregade	12	4200	Slagelse	Zealand Academy	Bredahlsgade	1	4200	Slagelse	Software Dev
1	work	Anna Jensen	anna@school.dk	12345678	Industrivej	8	4100	Ringsted	Zealand Academy	Bredahlsgade	1	4200	Slagelse	Software Dev
2	home	Mads Larsen	mads@school.dk	23456789	Parkvej	4	4000	Roskilde	Zealand Academy	Bredahlsgade	1	4200	Slagelse	IT Operations
2	work	Mads Larsen	mads@school.dk	23456789	Havnevej	19	4300	Holbæk	Zealand Academy	Bredahlsgade	1	4200	Slagelse	IT Operations

Problem: full_name, email, and phone depend only on user_id, not on the whole key.

These columns are repeated for each address row.

2NF version

Move columns that depend only on user_id into a separate table

users

user_id	full_name	email	phone	school_name	department_name
1	Anna Jensen	anna@school.dk	12345678	Zealand Academy	Software Dev
2	Mads Larsen	mads@school.dk	23456789	Zealand Academy	IT Operations

user_addresses

user_id	address_type	street_name	street_no	postal_code	city
1	home	Nørregade	12	4200	Slagelse
1	work	Industrivej	8	4100	Ringsted
2	home	Parkvej	4	4000	Roskilde
2	work	Havnevej	19	4300	Holbæk

school_locations

school_street_name	school_street_no	school_postal_code	school_city
Bredahlsgade	1	4200	Slagelse
Industrivej	8	4100	Ringsted

users still keeps school and department data. 3NF is not fixed yet.

Address rows stay in user_addresses; user-level data moves to users.

3NF — Start from the 2NF tables

A relation is in 3NF if it satisfies 2NF and additionally, there are no transitive dependencies.
In simpler terms, non-prime attributes
should not depend on other non-prime attributes.

2NF first

Use the 2NF tables as the starting point

school_locations

school_name	street_name	street_no	postal_code	city
Zealand Academy	Bredahlsgade	1	4200	Slagelse

Problem: city depends on postal_code, not on the key of these tables.

users

user_id	full_name	email	phone	school_name	department_name
1	Anna Jensen	anna@school.dk	12345678	Zealand Academy	Software Dev
2	Mads Larsen	mads@school.dk	23456789	Zealand Academy	IT Operations

user_addresses

user_id	address_type	street_name	street_no	postal_code	city
1	home	Nørregade	12	4200	Slagelse
1	work	Industrivej	8	4100	Ringsted
2	home	Parkvej	4	4000	Roskilde
2	work	Havnevej	19	4300	Holbæk

3NF version

Move only columns that depend on another non-key column

Only the transitive dependency is removed here.

users

user_id	full_name	email	phone	school_name	department_name
1	Anna Jensen	anna@school.dk	12345678	Zealand Academy	Software Dev
2	Mads Larsen	mads@school.dk	23456789	Zealand Academy	IT Operations

school_locations

school_name	street_name	street_no	postal_code
Zealand Academy	Bredahlsgade	1	4200

user_addresses

user_id	address_type	street_name	street_no	postal_code
1	home	Nørregade	12	4200
1	work	Industrivej	8	4100
2	home	Parkvej	4	4000
2	work	Havnevej	19	4300

postal_codes

postal_code	city
4200	Slagelse
4100	Ringsted
4000	Roskilde
4300	Holbæk

Now city is stored once per postal_code and reused from there.

Live normalization case

course_registrations

student_id	student_name	student_email	student_phone	student_address	course_id	course_name	teacher_id	teacher_name	room_id	room_name	room_building	meeting_days	school_id	school_name	school_address	department_id	department_name	grade
1	Anna Jensen	anna@school.dk	12 34 56 78	Nørregade 12, 4200 Slagelse	101	Databases	10	Mette Holm	201	A1	North	Mon, Wed	1	Zealand Academy	Bredahlsgade 1, 4200 Slagelse	11	Software Development	7
1	Anna Jensen	anna@school.dk	12 34 56 78	Nørregade 12, 4200 Slagelse	102	Systems Analysis	11	Lars Madsen	202	B2	North	Tue	1	Zealand Academy	Bredahlsgade 1, 4200 Slagelse	11	Software Development	10
2	Noah Berg	noah@school.dk	22 33 44 55	Parkvej 4, 4700 Næstved	101	Databases	10	Mette Holm	201	A1	North	Mon, Wed	1	Zealand Academy	Bredahlsgade 1, 4200 Slagelse	12	IT Operations	12

Look for

Repeated student, course, teacher, room, school, and department data. Address fields and meeting days also combine multiple facts in one place.

Goal

Use the live diagram to separate data into clearer tables with one main responsibility each and readable relationships between them.

Keep open

Do not lock the final table set in advance. Let the live normalization process decide which tables and relationships are needed.

EK ERHVERVSAKADEMI KØBENHAVN

What a join does

Use a simple domain first: employees and departments.

employees

employee_id	employee_name	department_id
1	Liva	10
2	Mads	20
3	Aya	
4	Omar	10

department_id ↔

departments

department_id	department_name
10	Sales
20	Support
30	Finance

Question

Which department does each employee belong to?

Join idea

Match the related values and return columns from both tables.

Readable output

Use names in the result, not only IDs.

INNER JOIN — keep matching rows only

Show each employee with the matching department name.

```
SELECT e.employee_name,
d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id =
d.department_id;
```

result

employee_name	department_name
Liva	Sales
Mads	Support
Omar	Sales

Included

Rows where employee.department_id matches a department row.

Excluded

Rows with no match, such as Aya with no department.

Use when

Only matched data should appear in the answer.

LEFT JOIN — keep all rows from the left table

Show all employees, even when no department is assigned.

```
SELECT e.employee_name,
d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id =
d.department_id;
```

result

employee_name	department_name
Liva	Sales
Mads	Support
Aya	NULL
Omar	Sales

Left side matters

Employees are on the left, so every employee row stays in the result.

No match

Missing department data becomes NULL instead of removing the row.

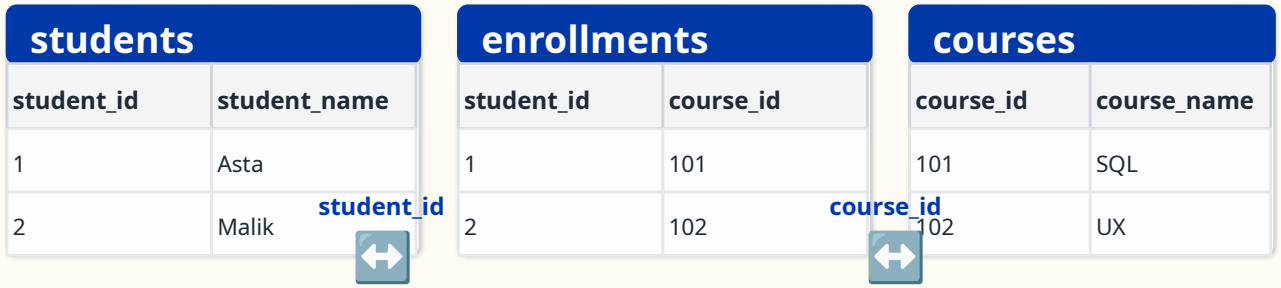
Use when

The answer must include unmatched rows too.

Same logic, new schema

The assignment uses a different domain. The join pattern stays the same.

- Read the question first.
- Find the tables that contain the needed facts.
- Find the matching keys.
- Return readable names instead of only IDs.
- Use LEFT JOIN when rows must stay even without a match.



Transfer example

employees.department_id = departments.department_id becomes enrollments.student_id = students.student_id and enrollments.course_id = courses.course_id.

Assignment prompts

1. Show which courses each student is enrolled in. Return the student name and the course name.

2. Show who teaches each course and where the course takes place. Return the course name, teacher name, room name, and building.

3. Show a full overview of each enrollment. Return the student name, course name, teacher name, room name, and grade.

4. Show all students, including those who are not enrolled in any course. Return the student name and the course name where available.

Optional prompt + result checks

Optional 5

Create one query that combines all relevant tables into one readable result. Return at least the student name, course name, teacher name, room name, building, and grade.

Before running the final answer

- Check whether the chosen tables really contain the needed facts.
- Check whether the join keys match the intended relationship.
- Check whether readable columns are shown in the result.
- Check whether INNER JOIN or LEFT JOIN fits the question.

Good answer

Readable, relevant, and based on the right relationships.